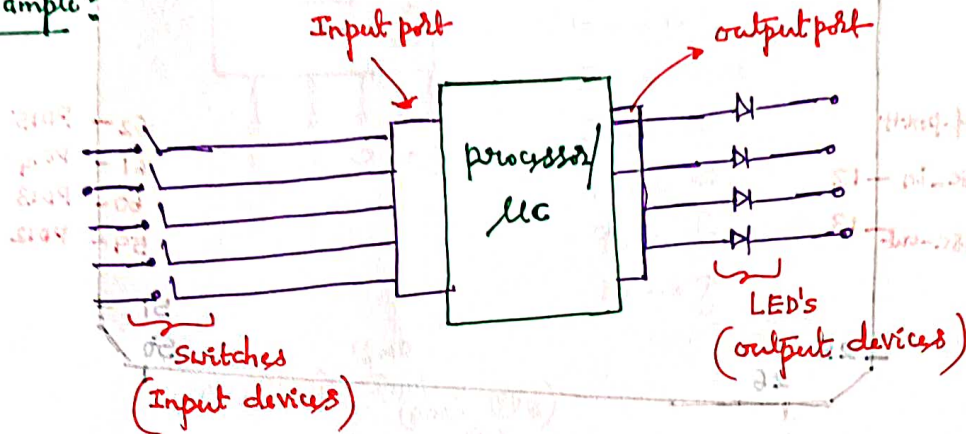


* GPIO [General purpose Input-output]: programming and Interfacing

→ I/O [Input/output] ports: are used by the processor/microcontroller to access the input and output devices.

Example:



→ types of I/Os:

1) General purpose I/O (GPIO): ports are used for Interfacing devices such as LEDs, switches, LCD, Keypad etc.

2) Special purpose I/O (SPIO): ports used for ADC, Timer, UART (universal asynchronous receiver transmitter) etc.

* General purpose I/O ports: In STM Arm chips

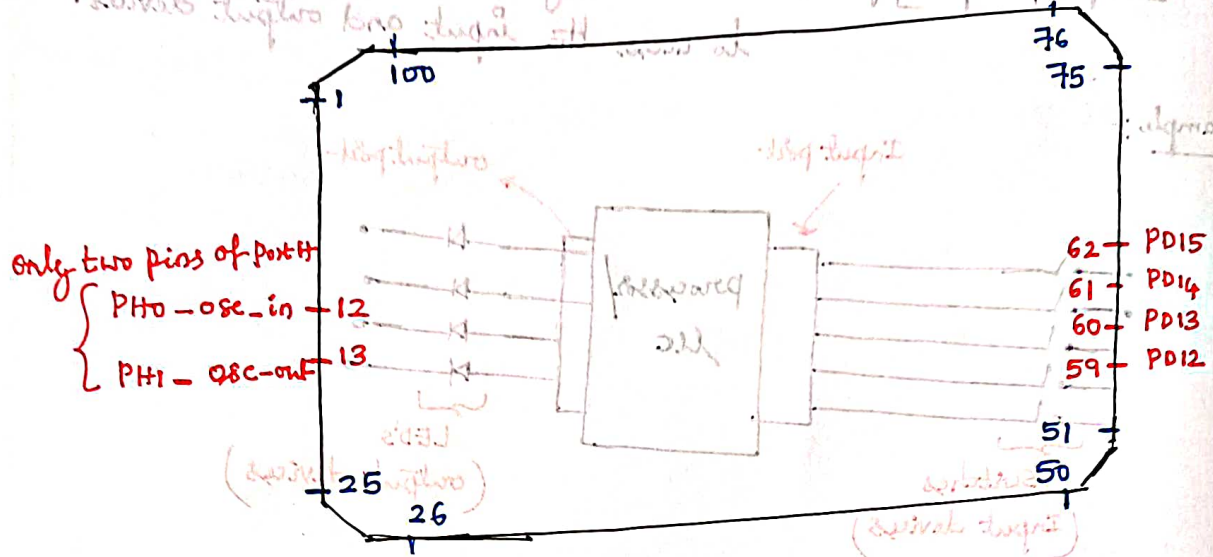
→ ports are, A, B, C, D, E, F, G & H. Totally eight ports.

→ Each port is having 16 pins. Ex: PA0 - PA15, PB0 - PB15 etc.

→ Each port is having 16 pins. Ex: PA0 - PA15, PB0 - PB15 etc.

NOTE: Not all 16 pins of each port are implemented in every chip.

* General Pin diagram :



→ The ARM chips having two types of buses.

1) APB (Advanced peripheral Bus) :

2) AHB (Advanced High performance Bus) :

→ AHB : Designed to Interface memory and fast I/O's directly to the CPU.

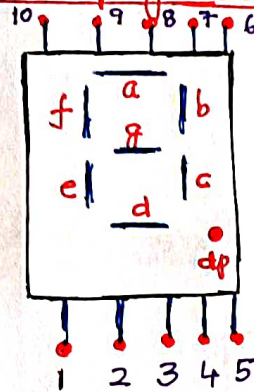
→ APB : Designed for low speed and low power consumption memory and peripherals.

→ The AHB bus much faster than APB.

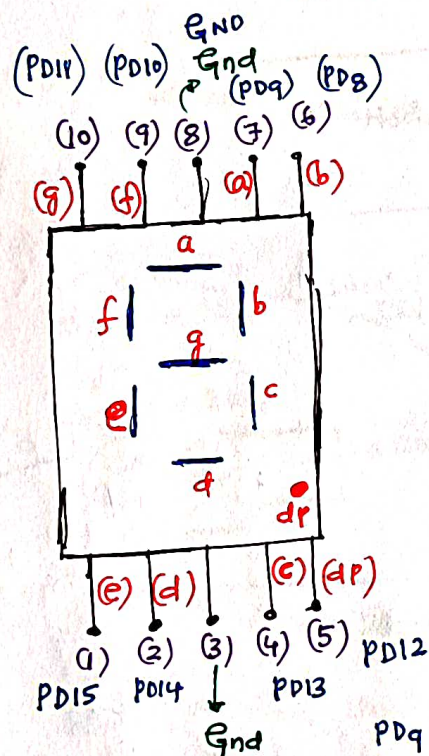
→ The AHB bus is much faster than APB. The AHB allows one clock cycle access to the memory and fast peripherals without any wait state.

→ The APB is slower and its access time is a minimum of 2 clock cycles.

* Seven Segment display Interfacing:



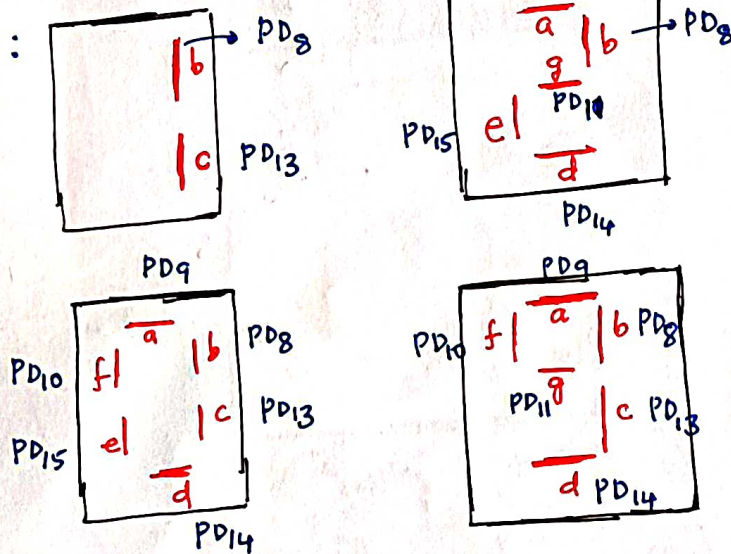
8 $\rightarrow$ Gnd
 7 $\rightarrow$ a
 6 $\rightarrow$ b
 5 $\rightarrow$ dp
 4 $\rightarrow$ c
 3 $\rightarrow$ Gnd
 2 $\rightarrow$ d
 1 $\rightarrow$ e
 9 $\rightarrow$ f
 10 $\rightarrow$ g



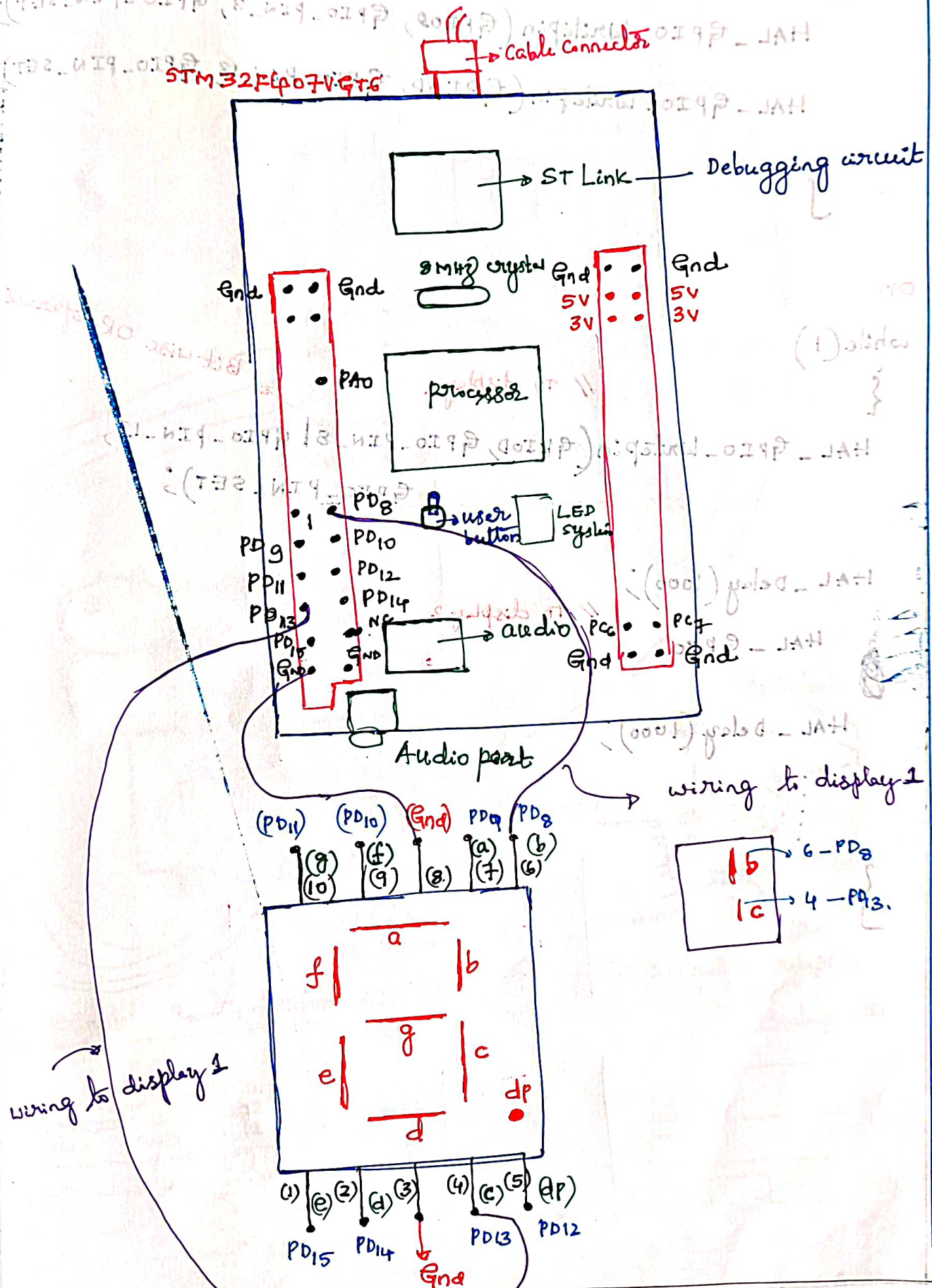
Configure the PD pins

PD15 -
 PD14
 PD13
 PD12
 PD11
 PD10
 PD9
 PD8

Example :



* Seven segment display Interfacing diagram:



while (1)

{

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET);

or.

while(1)

{

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8 | GPIO_PIN_13, GPIO_PIN_SET);

HAL_Delay(1000);

HAL_GPIO

HAL_Delay(1000);

}

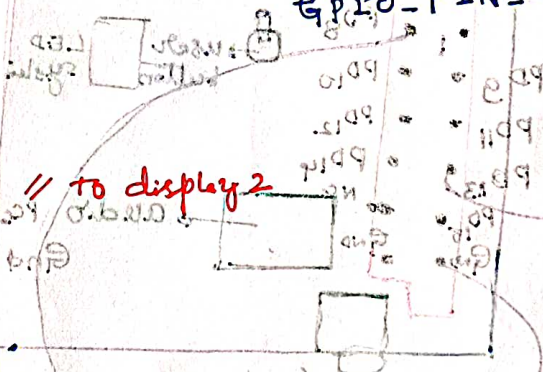
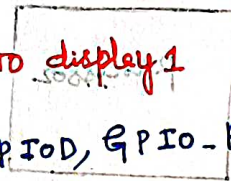
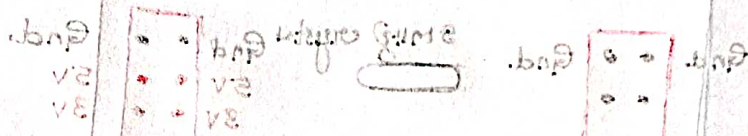
// to display 1

// to display 1

// to display 2

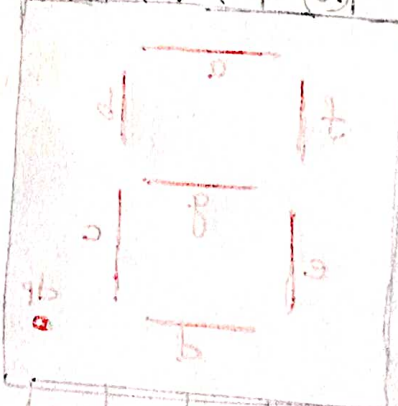
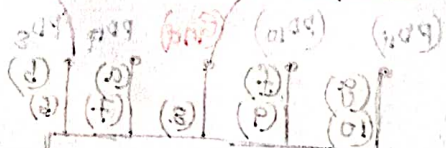
Bit wise OR operator

display triggered



display output

display triggered

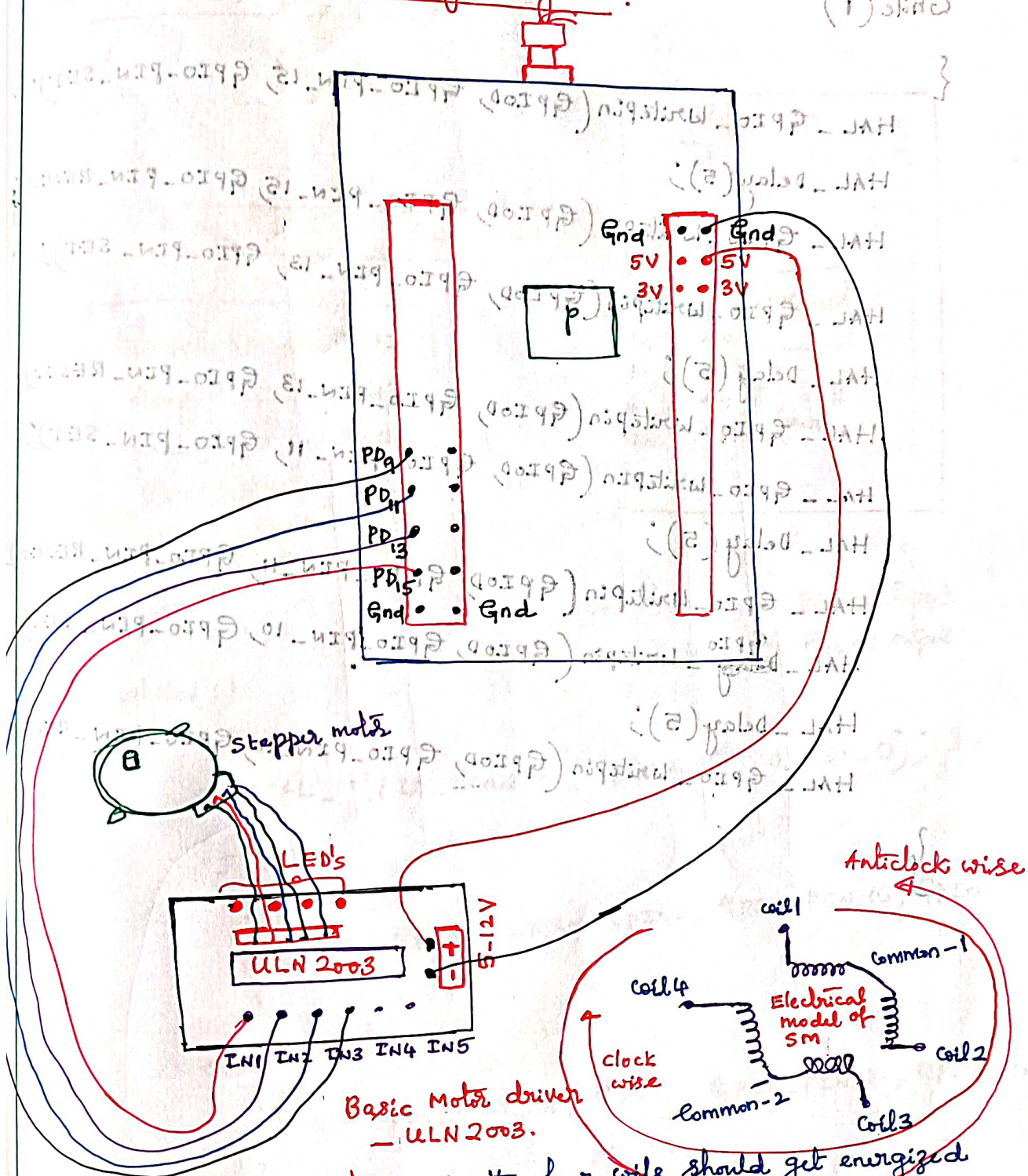


display triggered

* Stepper motor Interfacing diagram :

Dr. LBR

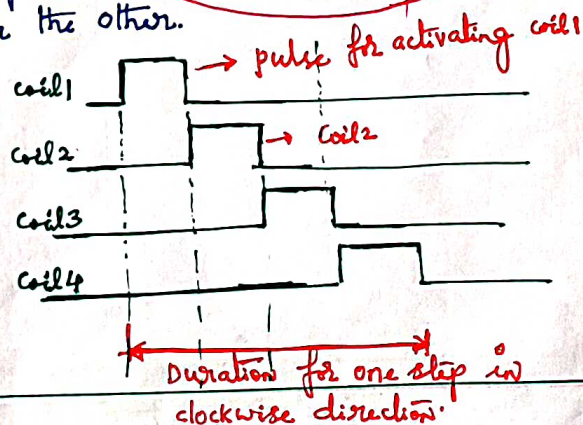
(1) anticlock



* principle of operation: All the four coils should get energized one after the other.

* Excitation Sequence:

Step	coil1	coil2	coil3	coil4
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1



while(1)

{

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_SET);

HAL_Delay(5);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_RESET);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET);

HAL_Delay(5);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET);

HAL_Delay(5);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET);

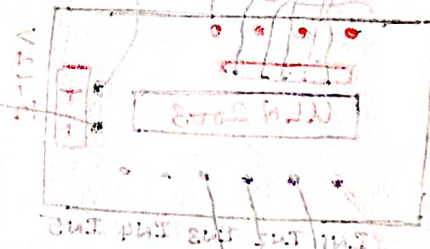
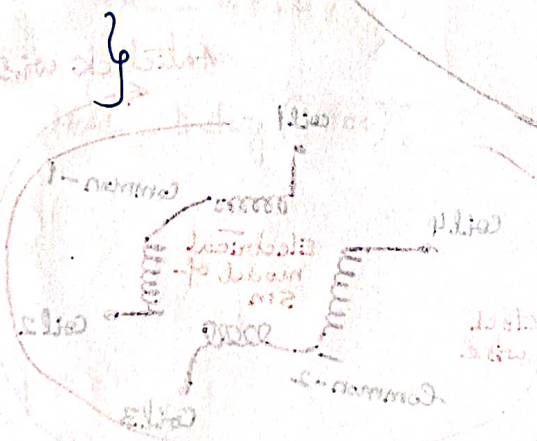
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET);

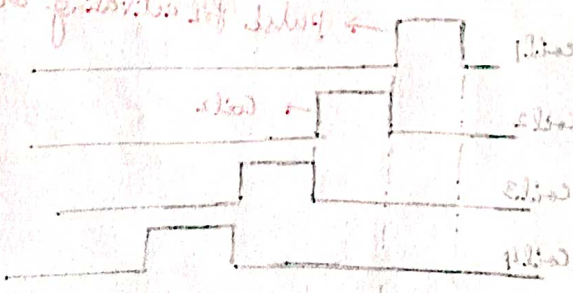
HAL_Delay(5);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET);

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET);



Principle of operation: All the four rows are active low. All the four columns are active low. All the four rows are active low. All the four columns are active low.

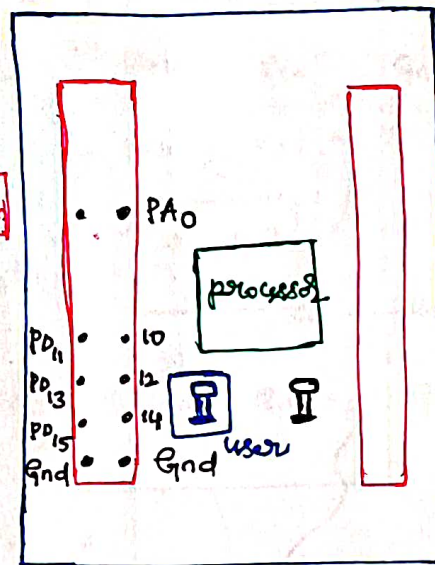


Col0	Col1	Col2	Col3	Row
0	0	0	1	1
0	0	1	0	2
0	1	0	0	3
1	0	0	0	4

* Setup push button Interfacing:

PA_0	ADC	PWM	user Button
------	-----	-----	-------------

Green LED	—	Pin12
Orange LED	—	Pin13
Red LED	—	Pin14
Blue LED	—	Pin15



NOTE: Configure PA0 as input
Configure PD14 as output

```
int a=0;
```

```
while(1)
```

```
{
```

```
    a = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);
```

```
    if (a)
```

```
    {
```

```
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET);
```

```
    }
```

```
else
```

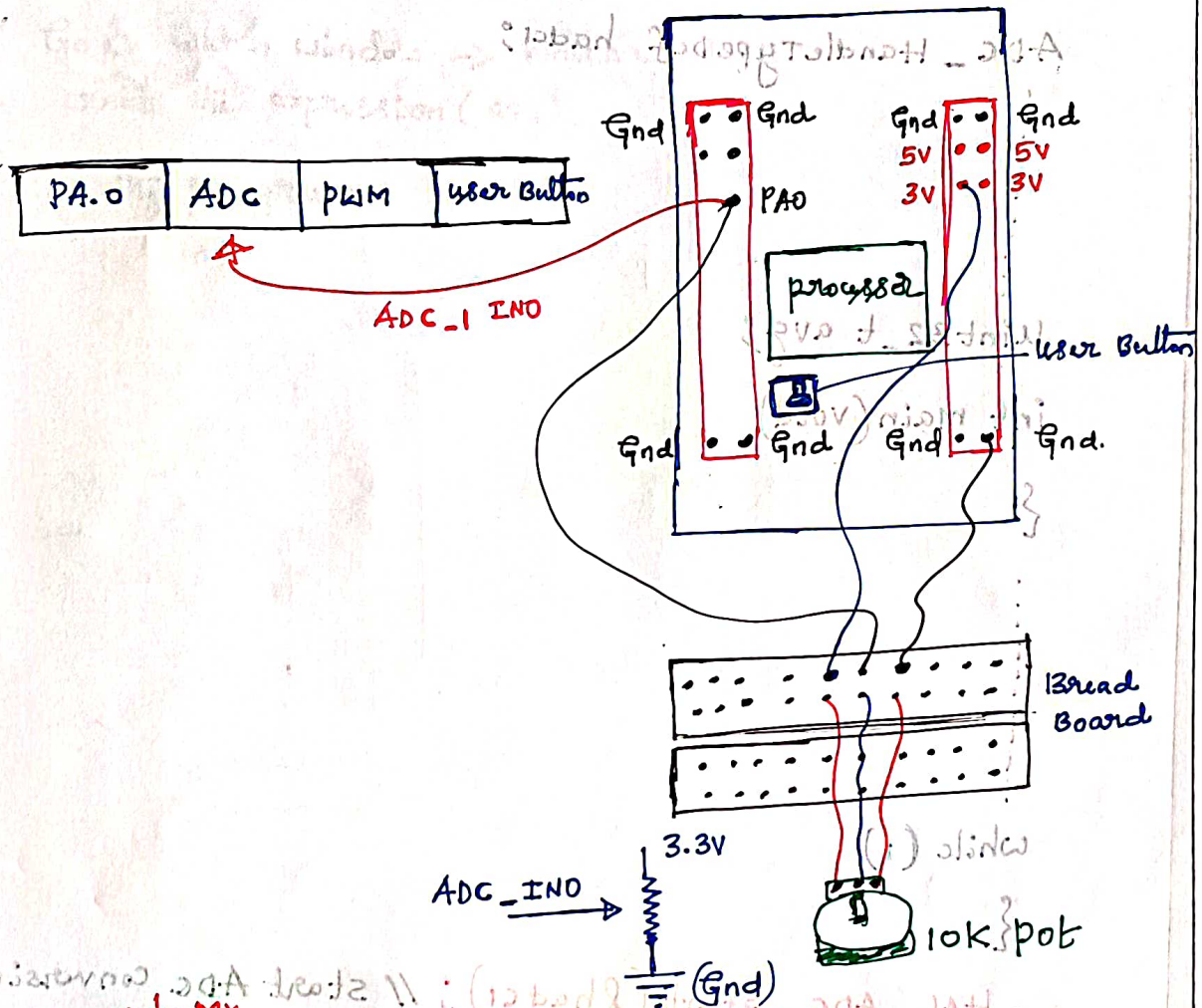
```
{
```

```
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_RESET);
```

```
}
```

```
}
```


* ADC: 10K pot Interfacing.



* ADC Setup:

- 1) Setup RCC & Setup sys; Setup ADC - select ADC1
ADC1 Mode and configuration: select IN0
NOTE
PA0 → IN0
PA1 → IN1

2) Setup pin configuration:

Setup pin PA0 to ADC1-IN0 - JAH = pvd

3) Setup project location & Name Generate the code

* ADC - Keil setup:

- 1) Create a variable (example arg) outside the main method
- 2) Check for the ADC handler name (hadc1)

3) Write code to start the ADC and perform conversion

ADC_HandleTypeDef hadc1;

uint32_t avg;

int main(void)

{

while (1)

HAL_ADC_Start(&hadc1); // start ADC conversion

if (HAL_ADC_PollForConversion(&hadc1, 1) == HAL_OK)

// Function calls peripheral (PA0) for every 1ms for conversion

{

avg = HAL_ADC_GetValue(&hadc1);

// converted digital value stores in avg variable

}

else

avg = 0;

}

}

4> Set Target → Save → Build → Load → Debug

5> Go to Watch window → Watch window
create the expression (avg)

6> Run the program.



- * Construction :
 - ↳ Build a voltage divider circuit
 - ↳ Reverse bias the Receiver
 - ↳ Take the output with respect to -ve of IR sensor and Gnd

* Working :

- * Working :
- When there is no flame, IR receives less IR Radiation. Therefore higher resistance causing higher voltage
 - When there is flame, IR receives higher radiation. Therefore lower resistance causing lower voltage.

NOTE: Same previous program (10K pot Integration) works here.

* ARM cortex M4/M3 - Memory mapping :-

	Vendor specific	0xFFFFFFF
PPB - External → ETM	private peripherals	0xE0100000 0xE00FFFFF 0xE0000000 0xDFFF FFFF ITM DWT FPB NVIC
	External Device	→ This region is used for external device memory. (off chip devices).
(IVN) External RAM	External RAM	0xA000 0000 0x9FFF FFFF IVN Reserved FPB DWT ITM
(TUD) Peripheral	Peripheral	0x60000000 0x5FFF FFFF → This region includes bit band and bit band alias areas.
(MTI) SRAM	SRAM	0x4000 0000 0x3FFF FFFF This executable Region is for data storage. Code can also be stored here. This region includes bit band and bit band alias areas.
(MTI) Code	Code	0x2000 0000 0x1FFF FFFF → This executable region is for program code. Data can also be stored here.

NOTE :-

Bit band alias

Bit band Region

Bit-band region is the region of memory and maps each bit in that region to an entire word in a second memory region (the bit-band alias region)

NOTE

- The ARM has a 32-bit address and in Hex addresses range from 00000000 to FFFFFFFF
- All of the I/O ports are mapped into the 4GB memory space.
- Code, data and all the I/O devices share the memory space and bytes for the data and code are different. This is very novel approach. [it means that all are accessed through different bytes but they are stored in the same space.]

private peripherals

0xE00FFFFF

0xE0000000

- private peripheral Bus - External
- private peripheral Bus - Internal

* private peripheral Bus - Internal

Reserved
NVIC
Reserved
FPB
DWT
ITM

- Nested Vectored Interrupt Controller (NVIC)
- Flashpatch and Breakpoint (FPB)
- Data Watchpoint and Trace (DWT)
- Instrumentation Trace macrocell (ITM)

* private peripheral Bus - External

ROM Table
External PPB
ETM
TPIU

- Embedded trace macrocell (ETM)
- Trace port Interface unit (TPIU)

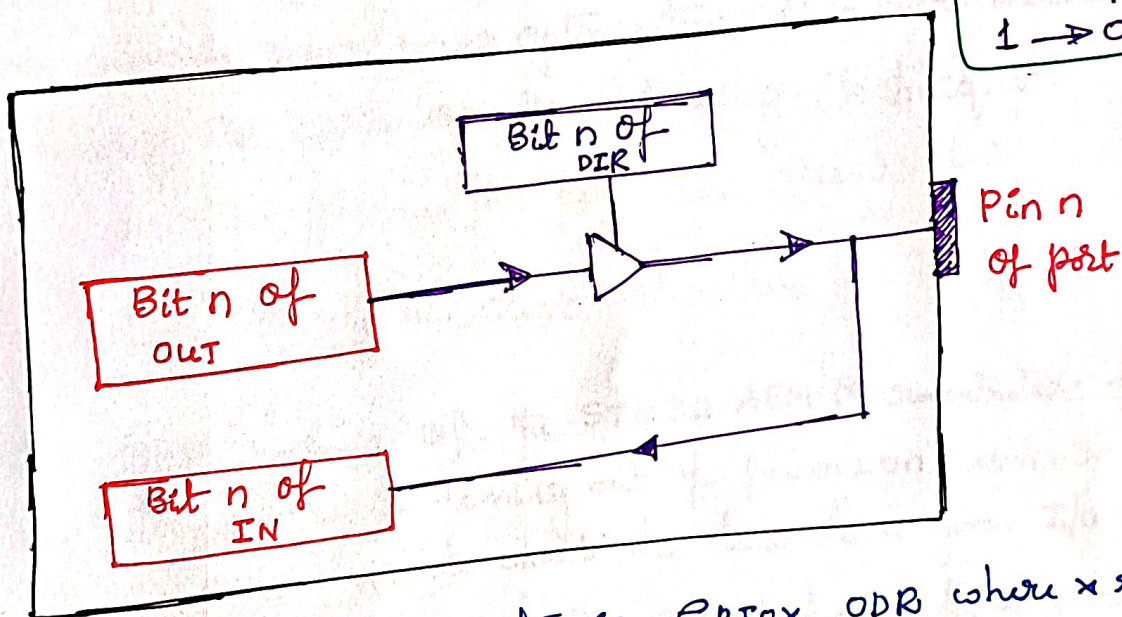
→ Generally, memory space is divided into the following three

- 1) code space:
- 2) Device space: External device
- 3) sharable memory space: External RAM is sharable and can be accessed by multiple devices.

* Data Register and Direction Register:-

- The above two registers are associated with each of I/O port.
- Direction Register: used to set the pin either input or output.
- Data Register: After the direction Register is configured, then data register is used to read the data from pin or write to the pin.

Example: when the pin is set as output by using direction register then the data in the data register is driven to the pins to which devices are connected.



Direction Register Configuration

0 → i/p

1 → o/p.

The syntax for output Register is GPIOx_ODR where x represents port name.

Ex: $\text{GPIOD} \rightarrow \text{ODR} = 0x0000F000;$

→ The direction Register is part of the GPIO_MODER (GPIO Mode Register).

→ For each GPIO pin, there are two corresponding bits in the MODER register.

→ The value of two bits are: $00 \rightarrow \text{Input}$ } configuration of port
 $01 \rightarrow \text{O/P}$ } pin as input/output.

Example: $\text{GPIOD} \rightarrow \text{MODER} = 0 \times 55000000$; Set pin to O/P mode.

* The clock Enable of the peripheral Registers :-

→ Before accessing the peripheral Registers, we need to check the clock enable registers.

→ To conserve power/power optimization, ARM microcontrollers having the either of the following Techniques.

* slowing down the peripheral circuit clock

* stopping the clocks altogether

→ The GPIO modules of the STM32 ARM Microcontrollers have the clock disabled coming out of power-on reset. So the programmers have to enable the clock to a given I/O port or peripheral before using it. This is done with a group of Registers belonging to RCC (Reset and clock control) register.

Ex: For the STM32F407VGT6 chip, the GPIO clock enable is controlled by lower bits of RCC_AHB1ENR (Advanced High performance Bus Enable Register)

$\text{RCC} \rightarrow \text{AHB1ENR} = 8$; To enable the clock to GPIO.

* 'C' program to Blink/Toggle all (Green, Orange, Red, Blue) LED's without using CubeMX Tool

```
#include "stm32f4xx.h"
```

```
void delay (int n);
```

```
int main(void)  
{
```

```
    RCC → AHBIENR |= 8; // Enable clock to GPIOD
```

```
    GPIOD → MODER |= 0x55000000; // set pin to O/P mode
```

```
    while (1)  
    {
```

```
        GPIOD → ODR ^= 0x0000F000; // turn on all LED's  
        delay (300);  
    }
```

```
}
```

```
void delay (int n)
```

```
{
```

```
    int i, j;
```

```
    for (j = 0; j < n; j++)
```

```
        for (i = 0; i < 3195; i++); // 16MHz system clock
```

```
}
```